SUMMARY: DATA STRUCTURES - I

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand the meaning of data structures and why we use them.
- Common data structures in Python
- Understand what lists are.
- Understand common methods and operations associated with lists.
- Understand the meaning of list comprehension
- Understand what tuples are.
- Understand common methods and operations associated with tuples.
- Understand Comparison between Lists and Tuples

KEY POINTS AND EXAMPLES:

Understand the meaning of data structures and why we use them:

data structures are ways to organize and store data in a computer so that it can be used efficiently. They provide a systematic way to manage and manipulate data, making it easier to perform operations like insertion, deletion, and retrieval.

We use data structures for several reasons:

- **Organization**: Data structures help arrange data in a neat and organized way, making it easy to find and work with.
- **Efficiency:** Different data structures are good at different tasks. Using the right one can make programs run faster and smoother.
- **Memory Management:** Data structures help use computer memory effectively, preventing waste and making sure everything fits neatly.
- **Flexibility**: Data structures can handle all sorts of data and let you do different things with it, making programs more versatile.
- **Abstraction**: Data structures hide the complicated details of how data is stored and managed, so programmers can focus on solving problems without getting bogged down.
- **Scalability**: With the right data structure, programs can handle more and more data without slowing down, keeping things running smoothly as they grow.

Common data structures in Python:

In Python, there are four common data structures:

1. **Lists:** Ordered collections of items, mutable.
2. **Tuples:** Ordered collections of items, immutable.
3. **Sets:** Unordered collections of unique items, mutable.
4. **Dictionaries:** Unordered collections of key-value pairs, mutable.

Understand what lists are

Lists in Python are one of the most commonly used data structures. They are versatile and can store a collection of items of different data types in single variable.

Lists are ordered, mutable (can be changed), and allow duplicate elements.

- **Ordered:** When we say that lists are ordered, it means that the items have a defined order, and that order will not change. If you add new items to a list, the new items will be placed at the end of the list.
- **Changeable(mutable):** The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.
- **Allow Duplicates:** Lists can have items with the same value

Creating Lists: we can create a list by enclosing elements within square brackets [], separated by commas.

```
list1 = [] #Empty list
list2 = [1, 2, 3] #List of integers
list3 = [1, "One", 3.4] #List with mixed data types
```

```
#Lists allow duplicate values:
thislist = ["apple", "banana", "cherry", "apple", "cherry"]
print(thislist)
```

The output will be:

```
['apple', 'banana', 'cherry', 'apple', 'cherry']
```

The list() Constructor: It is also possible to use the list() constructor when creating a new list. Using constructor we can convert one data structure to another.

```
thislist = list(("apple", "banana", "cherry")) # note the double
round-brackets
print(thislist)
```

The output will be:

```
['apple', 'banana', 'cherry']
```

```
my_string = "hello"
my_list = list(my_string)
print(my_list)
```

The output will be:

```
['h', 'e', 'l', 'l', 'o']
```

We can also use the type() function with lists

```
print(type(thislist))
```

The output will be:

```
<class 'list'>
```

A list can also have another list as an element. Such a list is called a Nested List.

```
list_ = ["One", [8, 4, 6], ['Three']] #Nested List
```

```
nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

<u>Understand common methods and operations associated with lists:</u>

Access Items: We can access items in a list using indexing or slicing.

```
my_list = ['apple', 'banana', 'cherry', 'date']
print(my_list[0])
print(my_list[:2])
print(my_list[-1])
print(my_list[::2])
print(my_list[1:4:2])
```

The output will be:

```
apple
['apple', 'banana']
date
['apple', 'cherry']
['banana', 'date']
```

```
nested_list = ["One", [8, 4, 6], ['Three']]
print(nested_list[1])
print(nested_list[2])
print(nested_list[1][0])
print(nested_list[2][0])
print(nested_list[1][1])
print(nested_list[1][2])
```

The output will be:

```
[8, 4, 6]
['Three']
8
Three
4
6
```

```
nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
element1= nested_list[0]
element2 = nested_list[1][1]
print(element1)
print(element2)
```

The output will be:

```
[1, 2, 3]
5
```

```
letters = ['a', 'b', 'c', 'd', 'e']
print(letters[-3:-1])
print(letters[-1:-3])
```

The output will be:

```
['c', 'd']
[]
```

As discussed previously, Python's slicing convention dictates that the starting index should come before the ending index.

List Length: To find the length of a list in Python, we can use the len() function.

```
my_list = ['apple', 'banana', 'cherry', 'date']
length = len(my_list)
print(length)
```

The output will be:

```
4
```

Finding minimum or maximum element in a list: To find the minimum or maximum element in a list, we can use the min() and max() functions in Python.

```
my_list = [10, 25, 5, 40, 15]
min_element = min(my_list)
max_element = max(my_list)
print( min_element)
print( max_element)
```

The output will be:

```
5
40
```

```
my_list = [10, 25, 5, 40, 15, True]
min_element = min(my_list)
```

```
max_element = max(my_list)
print(min_element)
print(max_element)
```

The output will be:

```
True
40
```

In Python, when using the min() and max() functions on a list containing elements of different types, the comparison is done based on the natural order of the elements. For numeric types (integers, floats), the comparison is straightforward, where smaller values come before larger values. However, when comparing numeric types with non-numeric types (like booleans), the comparison is done based on their respective numeric equivalents:

- False is equivalent to 0.
- True is equivalent to 1.

Note: We cannot compare two different data types like a string.Even though the data types are different , however since a float can be compared to a floating-point value

```
strings = ["abc", "abd", "abe"]
min_string = min(strings)
print( min_string)
max_string = max(strings)
print( max_string)
```

The output will be:

```
abc
abe
```

If the strings have the same length, Python compares them character by character until it finds a difference. If the strings have different lengths, Python compares them based on their lengths. The shorter string is considered smaller.

sum of all elements in a list: The sum() function in Python is used to calculate the sum of all elements in a list.

```
my_list = [1, 2, 3, 4, 5]
total = sum(my_list)
print(total)
```

The output will be:

```
15
```

Note: If we use a data type other than numbers in the list, the sum() function may not work as expected. The sum() function can only be used with iterables that contain numeric types such as integers or floats.

```
my_list = [10, 25, 5, 40, 15, True]
sum_element = sum(my_list)
print( sum_element)
```

The output will be:

```
96
```

In this case, my_list contains both integers and a boolean value (True). Since True evaluates to 1 in a numeric context, the sum() function treats it as such and includes it in the sum calculation. Therefore, the output of sum(my_list) would be the sum of all elements in the list, including the numeric value equivalents of True and False.

Change Item Value: To change the value of an item in a list in Python, we can directly assign a new value to the desired index.

```
my_list = [10, 20, 30, 40, 50]
# Change the value of the item at index 2
my_list[2] = 35
print(my_list)
```

The output will be:

```
[10, 20, 35, 40, 50]
```

Change a Range of Item Values: To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to

insert the new values.

```
thislist = ["apple", "banana", "cherry"]
thislist[1:3] = ["watermelon"]
print(thislist)
```

The output will be:

```
['apple', 'watermelon']
```

Repeating/Replicating Lists: Like strings, we can use * operator to replicate a list specified number of times.

```
l1 = [1,2, 10, 23]
print(l1*3)
```

The output will be:

```
[1,2,10,23,1,2,10,23,1,2,10,23]
```

Add List Items: Adding items to a list in Python can be done using various methods.

1. **Using the append() Method:** The append() method adds a single element to the end of the list.

Syntax:

```
list.append(item)
```

Example:

```
thislist = ["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist)
```

The output will be:

```
['apple', 'banana', 'cherry', 'orange']
```

2. **Using the extend() Method:** The extend() method adds from another list to the end of the list.

Syntax:

```
list.extend(list1)
```

Example:

```
l1=[1,2,3,4]
l2=[5,6,7,8]
l1.extend(l2) #Adds contents of l2 to l1 at the end
print(l1)
```

The output will be:

```
[1,2,3,4,5,6,7,8]
```

3. **Using the insert() Method:** The insert() method inserts a single element at a specified position in the list.

Syntax:

```
list.insert(index, item)
```

Example:

```
my_list = ['apple', 'banana', 'cherry']
my_list.insert(1, 'date')
print(my_list)
```

The output will be:

```
['apple', 'date', 'banana', 'cherry']
```

Note: If the index you provide is greater than the length of the list, Python will insert the element at the end of the list

4. **Using the + Operator:** The + operator concatenates two lists, creating a new list with the combined elements.

Syntax:

```
new_list = list1 + list2
```

Example:

```
list1 = ['apple', 'banana']
list2 = ['cherry', 'date']
new_list = list1 + list2
print(new_list)
```

The output will be:

```
['apple', 'banana', 'cherry', 'date']
```

Comparing Lists: In Python, we can compare lists using various comparison operators like ==, !=, <, <=, >, and >=. When comparing lists, Python checks if the elements at corresponding positions in the lists are equal or not.

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
print(list1 == list2) # Output: True
```

In this case, list1 and list2 have the same elements at the same positions. When comparing lists with the == operator, Python checks if the elements at corresponding positions in the lists are equal. Since list1 and list2 have the same elements [1, 2, 3] in the same order, the comparison list1 == list2 evaluates to True.

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
print(list1 != list2)  # Output: True
```

```
list1 = [1, 2, 3]
list2 = [1, 2, 4]
print(list1 < list2)  # Output: True (because 3 < 4)
```

```
list1 = [1, 2, 3]
list2 = [1, 2, 3, 4]
print(list1 < list2)  # Output: True (because list1 is shorter)
```

Notes:

- Lists are compared element-wise from left to right.
- If the corresponding elements are equal, the comparison proceeds to the next pair of elements.
- If one list is a prefix of the other, the shorter list is considered less.

- If all corresponding elements are equal and the lengths are the same, the lists are considered equal.

```
list1 = [1, 2, 3]
list2 = ['1', '2', '3']
print(list1 == list2) # Output: False
```

In the provided example, list1 contains integers [1, 2, 3], while list2 contains strings ['1', '2', '3']. Even though the values are the same, the types are different. The elements at corresponding positions in list1 and list2 are not equal in terms of both value and type. Therefore, the comparison list1 == list2 evaluates to False.

Remove List Items: Removing items from a list in Python can be achieved using various methods.

1. **Using the remove() Method:** The remove() method removes the first occurrence of a specified value from the list.

Syntax:

```
list.remove(value)
```

Example:

```
my_list = ['apple', 'banana', 'cherry', 'banana']
my_list.remove('banana')
print(my_list)
```

The output will be:

```
['apple', 'cherry', 'banana']
```

2. **Using the pop() Method:** The pop() method removes and returns the element at the specified index. If no index is provided, it removes the last element. If we do not specify the index, the pop() method removes the last item.

Syntax:

```
popped_element = list.pop([index])
```

Example:

```
my_list = ['apple', 'banana', 'cherry']
```

```
popped_element = my_list.pop(1)
print(popped_element)
print(my_list)
```

The output will be:

```
banana
['apple', 'cherry']
```

3. **Using the del Statement:** The del keyword also removes the specified index. If we do not specify the index, the del keyword also removes the list completely.

Syntax:

```
del list[index]
del list[start_index:end_index]
```

Example:

```
my_list = ['apple', 'banana', 'cherry']
del my_list[1]
print(my_list)
```

The output will be:

```
['apple', 'cherry']
```

```
thislist = ["apple", "banana", "cherry"]
del thislist
print(thislist)
```

The output will be:

```
NameError: name 'thislist' is not defined
```

this will cause an error because you have succsesfully deleted "thislist".

4. **Using the clear() Method:** The clear() method empties the list. The list still remains, but it has no content

Syntax:

```
list1.clear()
```

Example:

```
thislist = ["apple", "banana", "cherry"]
thislist.clear()
print(thislist)
```

The output will be:

```
[]
```

Sorting Lists: Sorting a list in Python can be done using the sort() method. We can sort a list in ascending or descending order.this method sort the list alphanumerically, ascending, by default. To sort descending, use the keyword argument reverse = True

```
my_list = [3, 1, 4, 2]
my_list.sort()
print(my_list)
```

The output will be:

```
[1,2,3,4]
```

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort()
print(thislist)
```

The output will be:

```
['banana', 'kiwi', 'mango', 'orange', 'pineapple']
```

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]
thislist.sort(reverse = True)
print(thislist)
```

The output will be:

```
['pineapple', 'orange', 'mango', 'kiwi', 'banana']
```

By default the sort() method is case sensitive, resulting in all capital letters being sorted before lower case letters.

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort()
print(thislist)
```

The output will be:

```
['Kiwi', 'Orange', 'banana', 'cherry']
```

we can use built-in functions as key functions when sorting a list. So if you want a case-insensitive sort function, use str.lower as a key function

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.sort(key = str.lower)
print(thislist)
```

The output will be:

```
['banana', 'cherry', 'Kiwi', 'Orange']
```

Joining Lists:
The join() method takes all items in an iterable and joins them into one string.

Syntax: string.join(iterable)

- **string**: This specifies the separator to be used between the elements.
- **iterable**: The collection of string elements to be joined.

Example 1:

```
words = ["Hello", "World", "Python"]
result = " ".join(words)
print(result)  # Output: Hello World Python
```

Example 2:

```
chars = ['P', 'y', 't', 'h', 'o', 'n']
```

```
result = "".join(chars)
print(result)  # Output: Python
```

Counting Occurrences in a List: The count() method allows you to determine how many times a specific element appears in a list.

Syntax:

```
count_ = list.count(element)
```

Example:

```
my_list = [1, 2, 3, 4, 2, 2, 5]
count_ = my_list.count(2)
print(count_)
```

The output will be:

```
3
```

Copying Lists: To create a copy of a list in Python, you can use different techniques

1. **Using the copy() Method:** The copy() method creates a shallow copy of the list. Changes made to the original list won't affect the copied list, and vice versa.

```
original_list = [1, 2, 3]
copied_list = original_list.copy()
```

2. **Using List Slicing:** List slicing can be used to create a copy of the entire list. This also creates a shallow copy.

```
original_list = [1, 2, 3]
copied_list = original_list[:]
```

3. **Using the list() Constructor:** The list() constructor can also be used to create a copy of the list.

```
original_list = [1, 2, 3]
copied_list = list(original_list)
```

Finding the Index of an Element in a List: The index() method allows you to determine the position of a specified element within a list. It returns the index of the first occurrence of the specified element in a list. If the element is not found in the list, it raises a ValueError.

Syntax:

```
index1 = list.index(element)
```

Example:

```
my_list = ['apple', 'banana', 'cherry', 'banana']
index = my_list.index('banana')
print(index)
```

The output will be:

```
1
```

Reversing the Order of Elements in a List: The reverse() method allows you to reverse the order of elements in a list.

Syntax:

```
list.reverse()
```

Example:

```
my_list = ['apple', 'banana', 'cherry']
my_list.reverse()
print(my_list)
```

The output will be:

```
['cherry', 'banana', 'apple']
```

Understand the meaning of list comprehension:

List comprehension is a concise way of creating lists in Python using a single line of code. List Comprehensions are used to make code shorter and more readable.(We will cover this topic in an upcoming sessions on loops and Control statements)

Understand what tuples are

A tuple is an ordered collection of elements, similar to a list. However, tuples are immutable, meaning they cannot be modified after creation. They can store a collection of items of different data types in single variable. Additionally, tuples allow duplicate values.

Creating Tuples: Tuples are created by enclosing elements within parentheses () separated by commas.

```
my_tuple = (1, 2, 3)
```

We can also create a tuple without parentheses by separating elements with commas.

```
my_tuple = 1, 2, 3
print(my_tuple)
```

The output will be:

```
(1, 2, 3)
```

The tuple() Constructor: It is also possible to use the tuple() constructor to make a tuple.

```
thistuple = tuple(["apple", "banana", "cherry"])
print(thistuple)
```

The output will be:

```
('apple', 'banana', 'cherry')
```

```
my_string = "hello"
my_tuple = tuple(my_string)
print(my_tuple)
```

The output will be:

```
('h', 'e', 'l', 'l', 'o')
```

Creating a tuple with a single item requires a trailing comma after the item inside parentheses.

```
my_tuple = (1,)
print(my_tuple) #output: (1,)
```

```
#NOT a tuple
thistuple = ("apple")
print(type(thistuple))  #output: <class 'str'>
```

We can also use the type() function with tuples.

```
tuple1 = ("abc", 20, True, "male")
print(type(tuple1))
```

The output will be:

```
<class 'tuple'>
```

A tuple can also contain another tuple as an element. Such a tuple is called a nested tuple.

```
nested_tuple = ((1, 2), ('a', 'b', 'c'), (True, False))
```

Understand common methods and operations associated with tuples:

Accessing tuple items: We can access items in a tuple using indexing or slicing.

```
_tuple = ('apple', 123, True, 3.14, [1, 2, 3])
print(_tuple[0])
print(_tuple[-1])
print(_tuple[1:4])
print(_tuple[:-1])
```

The output will be:

```
apple
[1, 2, 3]
```

```
(123, True, 3.14)
('apple', 123, True, 3.14)
```

Tuple Length: To find the length of a tuple in Python, we can use the len() function.

```
nested_tuple = ((1, 2), ('a', 'b', 'c'), (True, False))
length = len(nested_tuple)
print(length)
```

The output will be:

```
3
```

Finding minimum or maximum element in a tuple: To find the minimum or maximum element in a tuple, we can use the min() and max() functions in Python.

```
my_tuple = (10, 25, 5, 40, 15, True)
min_element = min(my_tuple)
max_element = max(my_tuple)
print(min_element)
print(max_element)
```

The output will be:

```
True
40
```

The same principle applies to tuples , boolean values (True and False) would be compared based on their numeric equivalents (False as 0 and True as 1) when present along with numeric types in tuples..

Note: We cannot compare two different data types like a string and a integer. Even though the data types are different , however since a float can be compared to a floating-point value

sum of all elements in a tuple: Just like lists, the sum() function in Python can also be used with tuples to calculate the sum of all numeric elements. However, the elements within the tuple must be numeric for this to work.

```
my_tuple = (1, 2, 3, 4, 5, True)
```

```
total = sum(my_tuple)
print(total)
```

The output will be:

```
16
```

Change Tuple Values: Since tuples are immutable, they do not have a built-in append() method, but there are other ways to add items to a tuple.

1. **Convert into a list:**

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.append("orange")
thistuple = tuple(y)
```

The output will be:

```
('apple', 'banana', 'cherry', 'orange')
```

2. **Add tuple to a tuple:**

```
thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y
print(thistuple)
```

The output will be:

```
('apple', 'banana', 'cherry', 'orange')
```

Tuples are immutable but a list within a tuple is mutable.

```
my_tuple = (1, 2, [3, 4])
my_tuple[2].append(5)
print(my_tuple)
output : (1, 2, [3, 4, 5])
```

The output will be:

```
(1, 2, [3, 4, 5])
```

While the elements inside a tuple cannot be modified after the tuple is created (immutable), the tuple itself can be reassigned to a new tuple. This doesn't modify the original tuple but creates a new one with the specified values.

```
t = (1, 2, 3)
t = (4, 5, 6)  # Reassignment of the tuple
print(t)  # Output: (4, 5, 6)
```

Remove Items: Tuples cannot be changed once they're created, so we can't remove items from them like we do with lists. However, we can convert the tuple into a list, remove items from the list, and then convert it back to a tuple if needed. This allows us to modify the data indirectly.

```
fruits = ("apple", "banana", "cherry", "banana", "cherry")
y = list(fruits)
y.remove("apple")
thistuple = tuple(y)
print(thistuple)
```

The output will be:

```
('banana', 'cherry', 'banana', 'cherry')
```

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.pop(0)
thistuple = tuple(y)
print(thistuple)
```

The output will be:

```
('banana', 'cherry')
```

The del keyword cannot be used to delete individual elements from a tuple because tuples are immutable in Python. However, we can use the del keyword to delete the entire tuple object itself.

```
my_tuple = (1, 2, 3, 4, 5)
del my_tuple
```

Unpacking a tuple: Unpacking a tuple in Python refers to the process of assigning the individual elements of a tuple to separate variables. It allows you to extract elements from a tuple and assign them to individual variables in a single step. This can be achieved by assigning the tuple to variables separated by commas, matching the number of variables to the number of elements in the tuple.

```
my_tuple = (1, 2, 3)
a, b, c = my_tuple
print(a)
print(b)
print(c)
```

The output will be:

```
1
2
3
```

Using an asterisk*: When the number of variables is less than the number of values in the tuple, adding an asterisk (*) to the variable name allows the remaining values to be assigned to the variable as a list.

```
my_tuple = (1, 2, 3, 4, 5)
a, *b, c = my_tuple
print(a)
print(b)
print(c)
```

The output will be:

```
1
[2,3,4]
```

```
5
```

Repeating/Replicating Tuples: we can use * operator to replicate a tuple specified number of times.

```
my_tuple = ('a', 'b', 'c')
replicated_tuple = my_tuple * 3
print(replicated_tuple)
```

The output will be:

```
('a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c')
```

Combining Tuples: To combine two or more tuples, we can utilize the + operator.

```
tuple1 = (1, 2, 3)
tuple2 = ('a', 'b', 'c')
combined_tuple = tuple1 + tuple2
print(combined_tuple)
```

The output will be:

```
(1, 2, 3, 'a', 'b', 'c')
```

```
t1 = (1, 2, 3)
t2 = 7
print(t1+t2)
```

The output will be:

```
TypeError: can only concatenate tuple (not "int") to tuple
```

This code will raise a TypeError because you cannot directly add an integer to a tuple. In Python, tuples can only be concatenated with other tuples. To add an integer to a tuple, we would need to create a new tuple containing the integer and concatenate it with the original tuple.

```
t1 = (1, 2, 3)
t2 = (7,)
print(t1+t2)
```

The output will be:

```
(1, 2, 3, 7)
```

(1, 2, 3, 7

Counting Occurrences in a Tuple: Counting occurrences in a tuple involves using the count() method, which returns the number of times a specified value appears in the tuple.

```
my_tuple = (1, 2, 3, 4, 2, 5, 2)
count_ = my_tuple.count(2)
print(count_)
```

The output will be:

```
3
```

Finding the Index of an Element in a Tuple: Finding the index of an element in a tuple involves using the index() method, which returns the index of the first occurrence of a specified value in the tuple.

```
my_tuple = (1, 2, 3, 4, 2, 5, 2)
index_of_2 = my_tuple.index(2)
print(index_of_2)
```

The output will be:

```
1
```

Understand Comparison between Lists and Tuples:

Lists	Tuples

Lists are mutable, meaning their elements can be modified after creation. You can add, remove, or change elements in a list.	Tuples, on the other hand, are immutable, meaning once they are created, their elements cannot be changed, added, or removed.
Lists are created using square brackets [ ].	Tuples are created using parentheses ( ).
Lists are slightly slower than tuples due to their mutability. When elements are added, removed, or modified in a list, it may require more memory operations.	Tuples are generally faster than lists because they are immutable. Once created, the elements in a tuple remain constant, resulting in better performance.
my_list = [1, 2, 3]	my_tuple = (1, 2, 3)
Due to their mutable nature, lists may consume more memory over time, especially as elements are added or removed.	Since tuples are immutable, they have a fixed size and occupy a constant amount of memory.

map(): The map() function in Python applies a given function to all items in an iterable (e.g., list, tuple) and returns a map object (an iterator).

```
map(function, iterable)
```

- **function:** The function to apply.
- **iterable:** The input data to process.

```
# Example - 1: Square each number in a list
numbers = [1, 2, 3, 4]
squared = map(lambda x: x**2, numbers)
print(list(squared)) # Converting map object to list
```

The output will be:

```
[1, 4, 9, 16]
```

```
# Example - 2: Get multiple numbers as input in one line
numbers = list(map(int, input().split()))
print("List of integers:", numbers)
```

- input() captures the user **input** as a single **string**.
- .split() splits the string into a list of substrings based on spaces.
- map(**int**, ...) converts each **substring to an integer.**
- list() converts the map object to a list of integers.

The output will be:

```
10 20 30 40 #Input from user - String
List of integers: [10, 20, 30, 40] #Each element is converted to
integer datatype using map(int,...)
```